

EMail

November 5, 2025

```
[1]: import pandas as pd
import numpy as np
```

```
[2]: from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, precision_score, recall_score, \
    f1_score, confusion_matrix
```

```
[3]: df = pd.read_csv("emails.csv")
```

```
[4]: print(df.head())
```

	Email No.	the	to	ect	and	for	of	a	you	hou	...	connevey	jay	\
0	Email 1	0	0	1	0	0	0	2	0	0	...	0	0	
1	Email 2	8	13	24	6	6	2	102	1	27	...	0	0	
2	Email 3	0	0	1	0	0	0	8	0	0	...	0	0	
3	Email 4	0	5	22	0	5	1	51	2	10	...	0	0	
4	Email 5	7	6	17	1	5	2	57	0	9	...	0	0	

	valued	lay	infrastructure	military	allowing	ff	dry	Prediction
0	0	0		0	0	0	0	0
1	0	0		0	0	0	1	0
2	0	0		0	0	0	0	0
3	0	0		0	0	0	0	0
4	0	0		0	0	0	1	0

[5 rows x 3002 columns]

```
[5]: print(df.shape)
```

(5172, 3002)

```
[6]: print(df.columns[:10])
```

```
Index(['Email No.', 'the', 'to', 'ect', 'and', 'for', 'of', 'a', 'you', 'hou'],
      dtype='object')
```

```
[9]: df.head(5)
```

```
[9]:   Email No.  the  to  ect  and  for  of   a  you  hou  ...  connevey  jay  \
0   Email 1    0  0   1   0   0  0   2   0   0  ...      0   0
1   Email 2    8 13  24   6   6  2 102   1  27  ...      0   0
2   Email 3    0  0   1   0   0  0   8   0   0  ...      0   0
3   Email 4    0  5  22   0   5  1  51   2  10  ...      0   0
4   Email 5    7  6  17   1   5  2  57   0   9  ...      0   0
```

```
      valued  lay  infrastructure  military  allowing  ff  dry  Prediction
0          0   0                0          0          0  0   0          0
1          0   0                0          0          0  1   0          0
2          0   0                0          0          0  0   0          0
3          0   0                0          0          0  0   0          0
4          0   0                0          0          0  1   0          0
```

[5 rows x 3002 columns]

```
[11]: df.isnull().sum()
```

```
[11]: Email No.      0
      the          0
      to          0
      ect         0
      and         0
      ..
      military    0
      allowing    0
      ff          0
      dry         0
      Prediction  0
      Length: 3002, dtype: int64
```

```
[ ]:
```

```
[12]: X = df.iloc[:,1:3001] # word frequency features
      X
```

```
[12]:   the  to  ect  and  for  of   a  you  hou  in  ...  enhancements  \
0     0  0   1   0   0  0   2   0   0  0  ...      0
1     8 13  24   6   6  2 102   1  27 18  ...      0
2     0  0   1   0   0  0   8   0   0  4  ...      0
3     0  5  22   0   5  1  51   2  10  1  ...      0
4     7  6  17   1   5  2  57   0   9  3  ...      0
...  ... ..  ... ..  ... ..  ... ..  ... ..  ...
5167  2  2   2   3   0  0  32   0   0  5  ...      0
5168 35 27  11   2   6  5 151   4   3 23  ...      0
```

5169	0	0	1	1	0	0	11	0	0	1	...	0
5170	2	7	1	0	2	1	28	2	0	8	...	0
5171	22	24	5	1	6	5	148	8	2	23	...	0

	connevey	jay	valued	lay	infrastructure	military	allowing	ff	dry
0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	1	0
2	0	0	0	0	0	0	0	0	0
3	0	0	0	0	0	0	0	0	0
4	0	0	0	0	0	0	0	1	0
...	...	...	...	...	...	...	...	...	...
5167	0	0	0	0	0	0	0	0	0
5168	0	0	0	0	0	0	0	1	0
5169	0	0	0	0	0	0	0	0	0
5170	0	0	0	0	0	0	0	1	0
5171	0	0	0	0	0	0	0	0	0

[5172 rows x 3000 columns]

```
[13]: Y = df.iloc[:, -1].values # 1 = spam, 0 = not spam
      Y
```

```
[13]: array([0, 0, 0, ..., 1, 1, 0], shape=(5172,))
```

```
[14]: # Split data
      X_train, X_test, y_train, y_test = train_test_split(X, Y, test_size=0.25,
      random_state=42)
```

```
[15]: from sklearn.metrics import classification_report, confusion_matrix
```

```
[16]: # ----- Support Vector Machine -----
      svc = SVC(C=1.0, kernel='rbf', gamma='auto')
      svc.fit(X_train, y_train)
      svc_pred = svc.predict(X_test)
```

```
[17]: print("Accuracy:", accuracy_score(y_test, svc_pred))
```

Accuracy: 0.8932714617169374

```
[18]: print("Precision:", precision_score(y_test, svc_pred))
```

Precision: 0.8734567901234568

```
[19]: print("Recall:", recall_score(y_test, svc_pred))
```

Recall: 0.7447368421052631

```
[20]: print("F1 Score:", f1_score(y_test, svc_pred))
```

F1 Score: 0.8039772727272727

```
[21]: print("\nConfusion Matrix:\n", confusion_matrix(y_test, svc_pred))
```

Confusion Matrix:

```
[[872  41]
 [ 97 283]]
```

```
[22]: print("\nClassification Report:\n", classification_report(y_test, svc_pred))
```

Classification Report:

	precision	recall	f1-score	support
0	0.90	0.96	0.93	913
1	0.87	0.74	0.80	380
accuracy			0.89	1293
macro avg	0.89	0.85	0.87	1293
weighted avg	0.89	0.89	0.89	1293

```
[23]: # ----- K-Nearest Neighbors -----
knn = KNeighborsClassifier(n_neighbors=7)
knn.fit(X_train, y_train)
knn_pred = knn.predict(X_test)
```

```
[24]: print("KNN Accuracy:", accuracy_score(y_test, knn_pred))
print("KNN Precision:", precision_score(y_test, knn_pred))
print("KNN Recall:", recall_score(y_test, knn_pred))
print("KNN F1 Score:", f1_score(y_test, knn_pred))
print("\nKNN Confusion Matrix:\n", confusion_matrix(y_test, knn_pred))
print("\nKNN Classification Report:\n", classification_report(y_test, knn_pred))
```

KNN Accuracy: 0.8662026295436969
KNN Precision: 0.7368421052631579
KNN Recall: 0.8473684210526315
KNN F1 Score: 0.7882496940024479

KNN Confusion Matrix:

```
[[798 115]
 [ 58 322]]
```

KNN Classification Report:

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

0	0.93	0.87	0.90	913
1	0.74	0.85	0.79	380
accuracy			0.87	1293
macro avg	0.83	0.86	0.85	1293
weighted avg	0.87	0.87	0.87	1293

[]: